

Resume Shortlisting using Classical AI Techniques

Classical Artificial Intelligence Workflow

SECTION 1

Project Objective

Objective

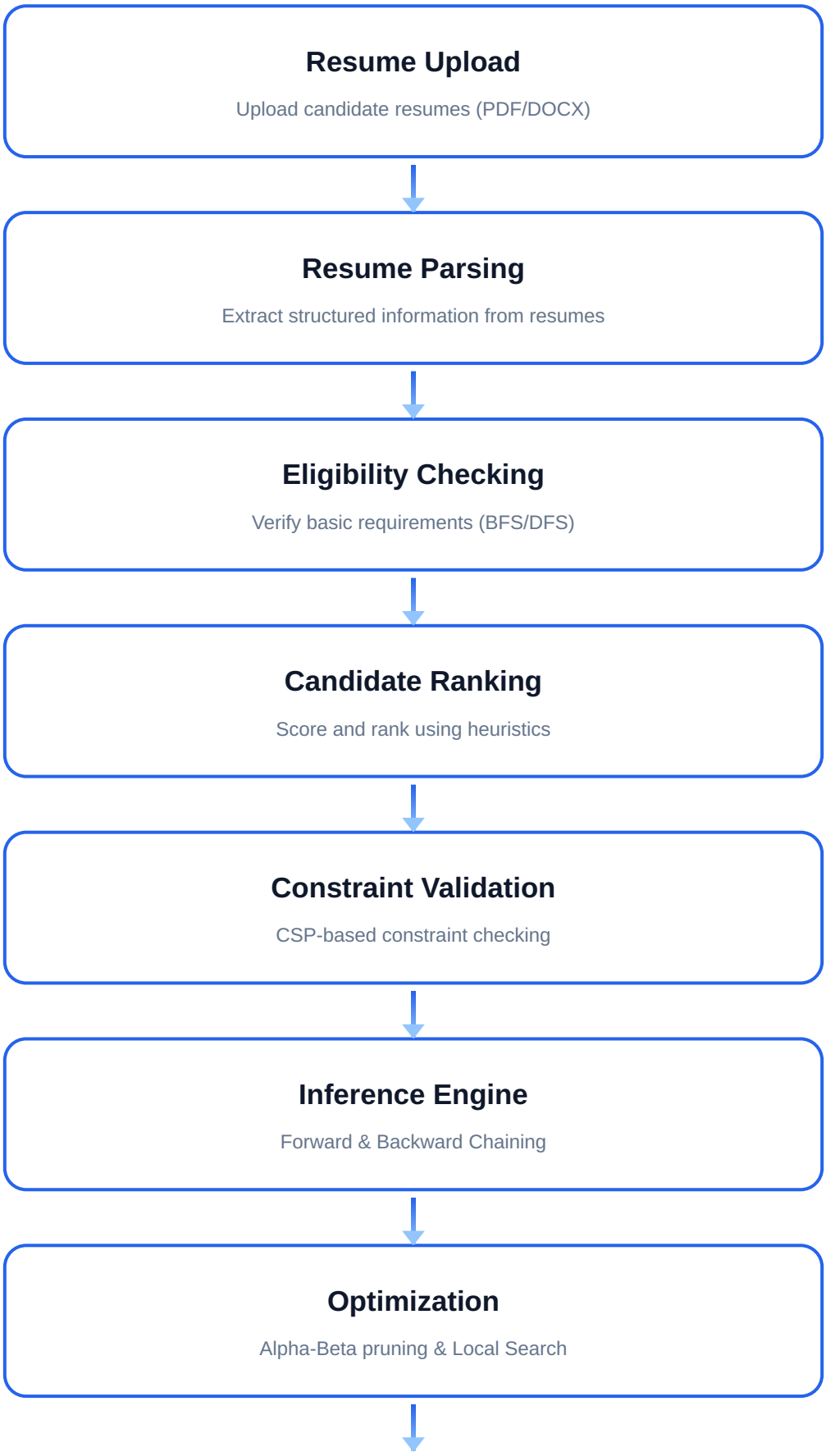
The primary objective of this project is to develop a **Resume Shortlisting System** using **Classical Artificial Intelligence** techniques instead of Machine Learning approaches. The system leverages rule-based AI algorithms to automate the initial screening and ranking of job applicants based on their resumes.

The system is designed to perform the following core functions: **parse uploaded resumes** to extract critical information such as education, skills, experience, and certifications; **check basic eligibility** against predefined company requirements using uninformed search techniques; **rank candidates** using heuristic search functions that weigh multiple criteria; **validate hiring constraints** using Constraint Satisfaction Problems to ensure only compliant candidates proceed; **eliminate weak candidates early** using Alpha-Beta pruning; and ultimately **produce a final ranked candidate list** ready for HR review. Every algorithm used in this pipeline is a classical AI technique, making the system transparent, explainable, and free from the opacity associated with machine learning models. This report explains every algorithm in detail and demonstrates exactly where it is applied within the workflow.

SECTION 2

System Overview

The system follows a sequential pipeline architecture where each stage processes the output of the previous stage. The complete system architecture is as follows:


Resume Upload
Upload candidate resumes (PDF/DOCX)
Resume Parsing
Extract structured information from resumes
Eligibility Checking
Verify basic requirements (BFS/DFS)
Candidate Ranking
Score and rank using heuristics
Constraint Validation
CSP-based constraint checking
Inference Engine
Forward & Backward Chaining
Optimization
Alpha-Beta pruning & Local Search

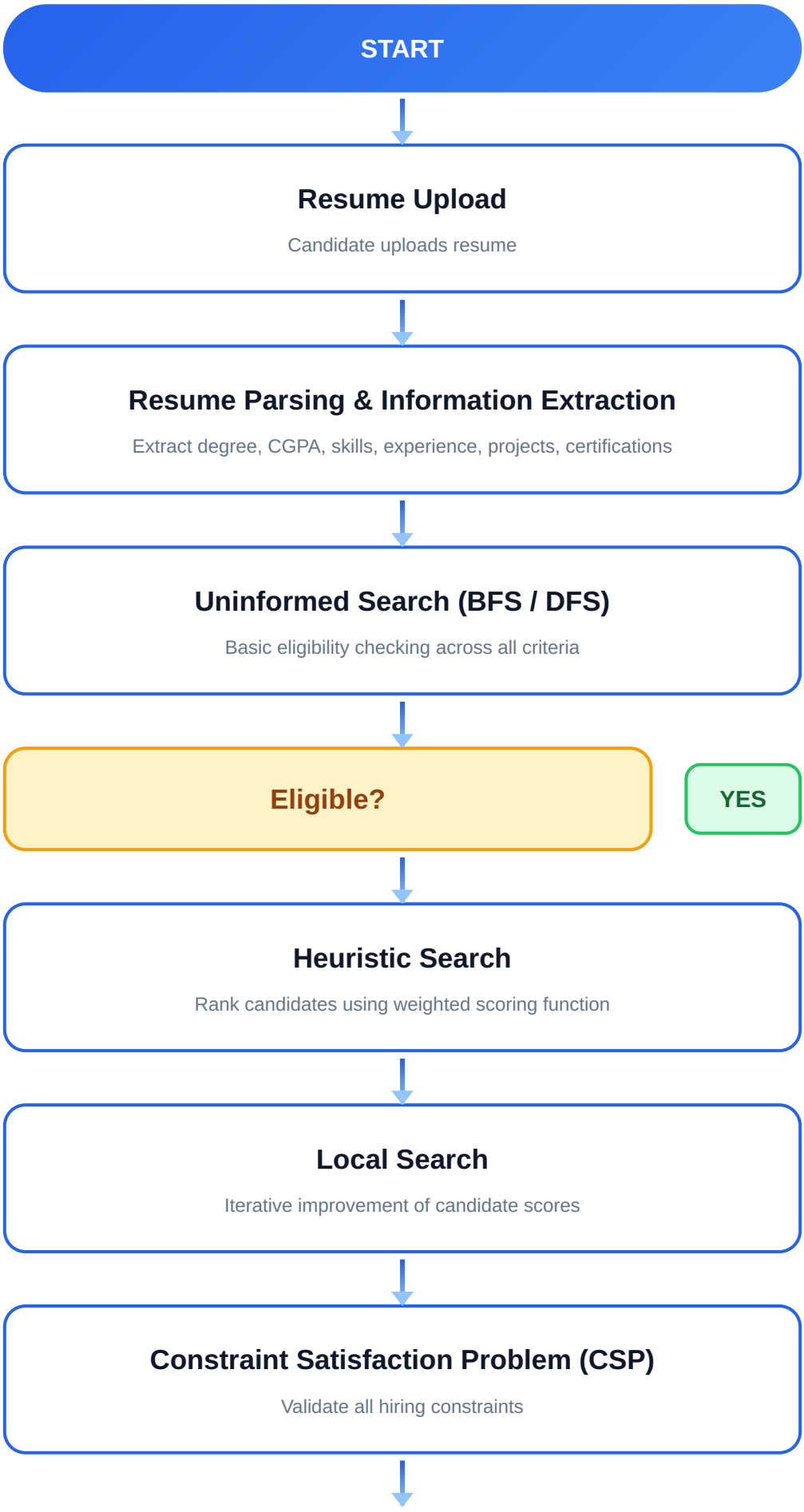
Final Ranking

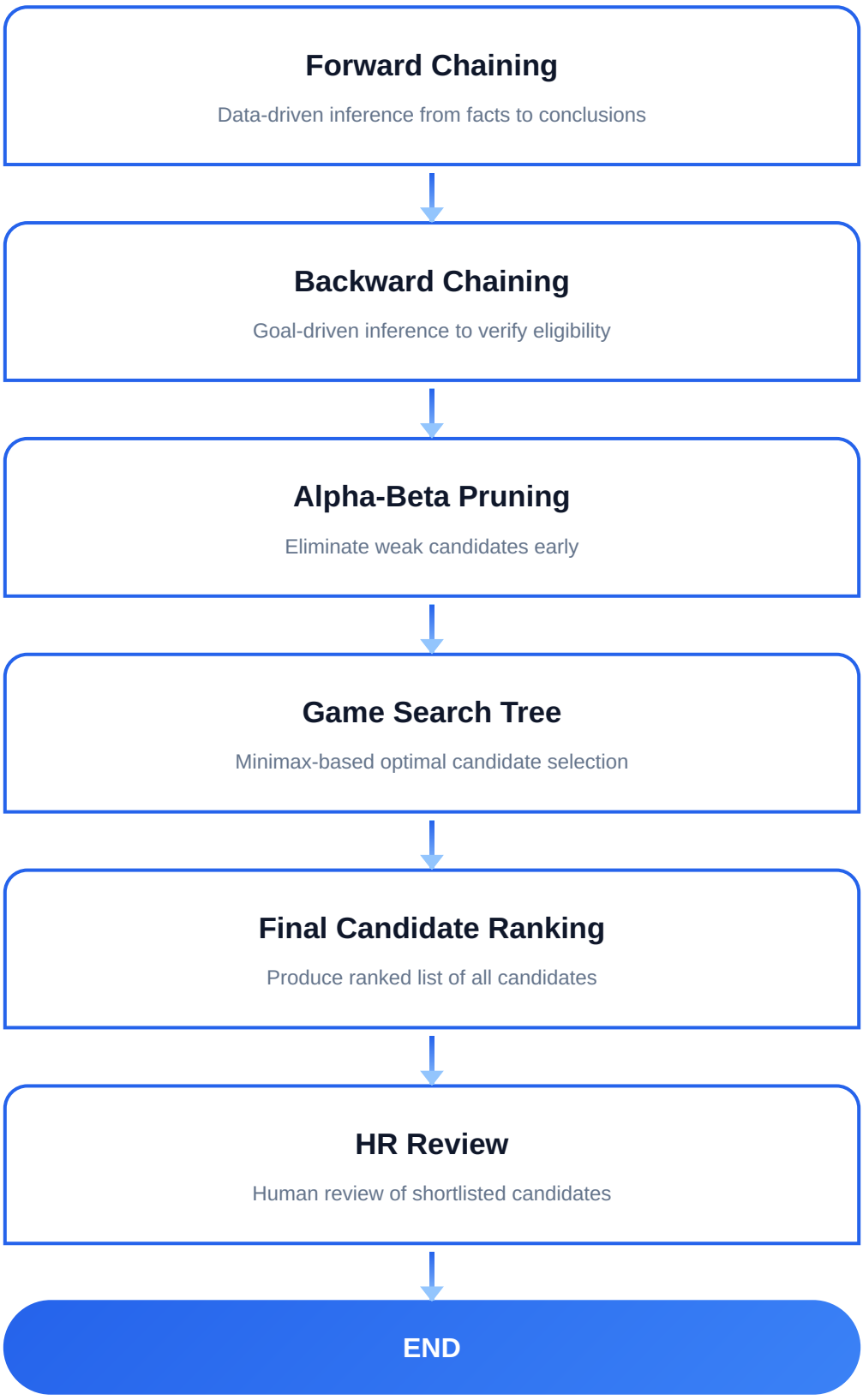
Produce final ranked candidate list

SECTION 3

Workflow Diagram

The following diagram illustrates the complete end-to-end workflow of the resume shortlisting system, showing every classical AI technique and where it is applied.


START
Resume Upload
Candidate uploads resume
Resume Parsing & Information Extraction
Extract degree, CGPA, skills, experience, projects, certifications
Uninformed Search (BFS / DFS)
Basic eligibility checking across all criteria
Eligible?
YES
Heuristic Search
Rank candidates using weighted scoring function
Local Search
Iterative improvement of candidate scores
Constraint Satisfaction Problem (CSP)
Validate all hiring constraints


Forward Chaining
Data-driven inference from facts to conclusions
Backward Chaining
Goal-driven inference to verify eligibility
Alpha-Beta Pruning
Eliminate weak candidates early
Game Search Tree
Minimax-based optimal candidate selection
Final Candidate Ranking
Produce ranked list of all candidates
HR Review
Human review of shortlisted candidates
END

SECTION 4

Resume Parsing

What is Resume Parsing?

Resume parsing is the process of extracting structured information from unstructured resume documents. It involves reading the raw text of a resume and identifying key fields such as education, skills, experience, projects, and certifications, converting them into a machine-readable format that the AI system can process.

Example Resume

Rahul

DEGREE **B.Tech**	CGPA **8.6**
SKILLS **Python, Java, SQL**	EXPERIENCE **2 Years**
PROJECTS **Hospital Management System**	CERTIFICATION **AWS**

How It Becomes Structured Data

Once parsed, the resume data is converted into structured key-value pairs that the system can query. Each extracted field is mapped to a predefined category, enabling the subsequent AI algorithms to operate on clean, standardized data rather than raw text.

Field	Extracted Value	Data Type
Name	Rahul	String
Degree	B.Tech	String
CGPA	8.6	Float
Skills	Python, Java, SQL	List[String]
Experience	2 Years	Integer
Projects	Hospital Management System	List[String]
Certification	AWS	List[String]

SECTION 5

Uninformed Search (BFS / DFS)

Purpose: Basic Eligibility Checking

Why BFS is used: Breadth-First Search explores all criteria level by level, ensuring that every eligibility requirement (Degree, CGPA, Experience, Skills) is checked systematically before making a decision. This prevents a candidate from passing one criterion while failing another.

Why DFS is used: Depth-First Search follows a single path of criteria checking deeply, which can quickly eliminate candidates who fail an early requirement without needing to check all remaining criteria.

Eligibility Flow



Complexity

$$O(V + E)$$

Example

COMPANY REQUIREMENTS

CANDIDATE (RAHUL)

Criteria	Required	Candidate	Result
Degree	B.Tech	B.Tech	**Pass**
CGPA	> 7.5	8.6	**Pass**
Experience	> 1 year	2 years	**Pass**
Skills	Python, Java, SQL	Python, Java, SQL	**Pass**

SECTION 6

Heuristic Search

Purpose: Ranking Resumes

Heuristic search assigns a weighted score to each candidate based on multiple criteria, enabling the system to rank them effectively. The heuristic function combines skill matching, experience, project quality, and certifications into a single score.

Heuristic Function

$$h(n) = w_1S + w_2E + w_3P + w_4C$$

Variables

Variable	Description	Weight
S	Skill Score	40%
E	Experience Score	30%
P	Project Score	20%
C	Certification Score	10%

Score Calculations

Skill Score

```
Skill Score = (Matched Skills / Required Skills) × 100
```

Example: Required Skills: Python, Java, SQL, HTML, CSS (5 total). Candidate has: Python, Java, SQL, HTML (4 matched).

```
Skill Score = (4 / 5) × 100 = 80
```

Experience Score

```
Experience Score = (Candidate Experience / Required Experience) × 100
```

Example: Required: 5 years. Candidate: 4 years.

```
Experience Score = (4 / 5) × 100 = 80
```

Project Score

Projects are scored based on their scale and significance:

- **Major Project** = 100 points
- **College Major Project** = 80 points
- **Mini Project** = 60 points

Certification Score

```
Certification Score = (Certificates Obtained / Required Certificates) × 100
```

Example: Required: AWS, Azure, Google, Oracle (4). Candidate has: AWS (1).

```
Certification Score = (1 / 4) × 100 = 25
```

Final Heuristic Score Calculation

Component	Score	Weight	Weighted Value
Skills	80	0.4	0.4 × 80 = 32
Experience	80	0.3	0.3 × 80 = 24
Projects	80	0.2	0.2 × 80 = 16
Certification	25	0.1	0.1 × 25 = 2.5
		Final Score	**74.5**

SECTION 7

Local Search

Why Local Search is Needed

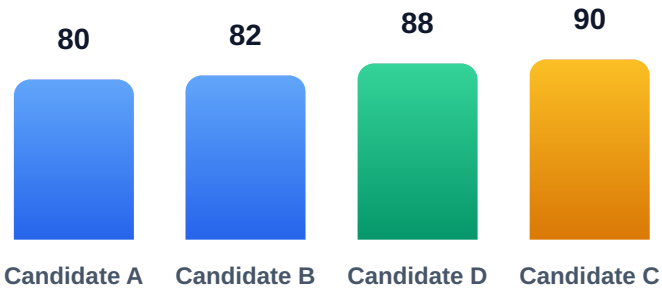
Local search is needed to iteratively improve the candidate ranking by exploring neighboring solutions. Instead of evaluating every possible combination, the algorithm starts with an initial ranking and replaces the current solution with a better neighboring candidate until the highest score is reached. This makes the process efficient for large candidate pools.

Example: Iterative Improvement

CANDIDATE A	CANDIDATE B
80	**82**

CANDIDATE C	CANDIDATE D
90	**88**

Optimization Diagram



The algorithm starts with Candidate A (80), compares with neighboring Candidate B (82) which is better, then continues to explore and finds Candidate D (88), and finally reaches the best candidate C (90). The current best solution is continually replaced until no better neighbor exists.

SECTION 8

Constraint Satisfaction Problem (CSP)

Constraint Satisfaction

A Constraint Satisfaction Problem ensures that only candidates who satisfy all hiring constraints proceed in the pipeline. CSP models the hiring process using variables, domains, and constraints to systematically filter candidates.

CSP Components

VARIABLES

Degree, Skills, Experience, CGPA, Location

DOMAINS

Possible values for each variable

Constraints

- Degree = B.Tech
- CGPA > 7.5
- Python Required
- Experience > 1 year

CSP Table

Variable	Domain	Constraint
Degree	B.Tech, M.Tech, B.Sc, BCA	Degree = B.Tech

Variable	Domain	Constraint
CGPA	5.0 – 10.0	CGPA > 7.5
Skills	Python, Java, C++, SQL, etc.	Python Required
Experience	0 – 20 years	Experience > 1 year

CSP guarantees that only valid candidates proceed because it systematically checks every variable against its constraints. If a candidate fails even a single constraint, they are eliminated from the pipeline before more computationally expensive algorithms are applied.

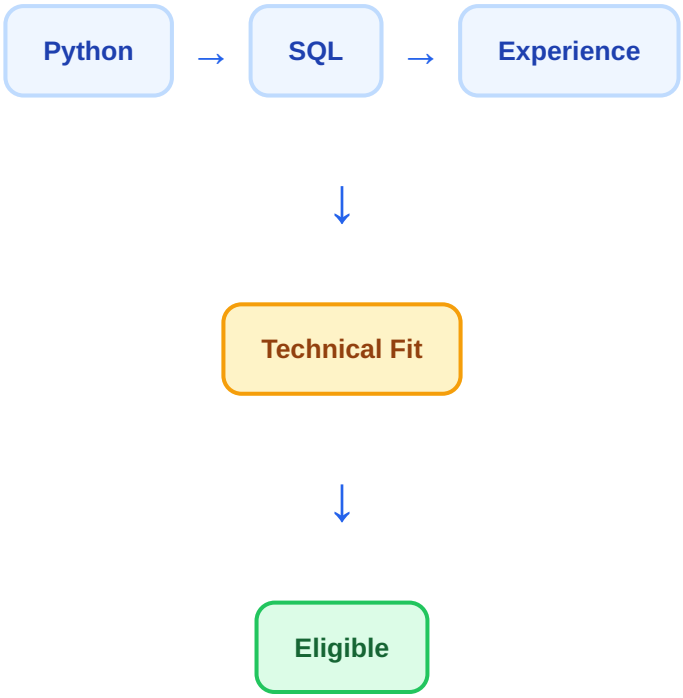
SECTION 9

Forward Chaining

Data-Driven Inference

Forward chaining is a data-driven inference method that starts from known facts and applies rules to derive new conclusions. It moves from facts to conclusions in a bottom-up manner.

Rule Diagram



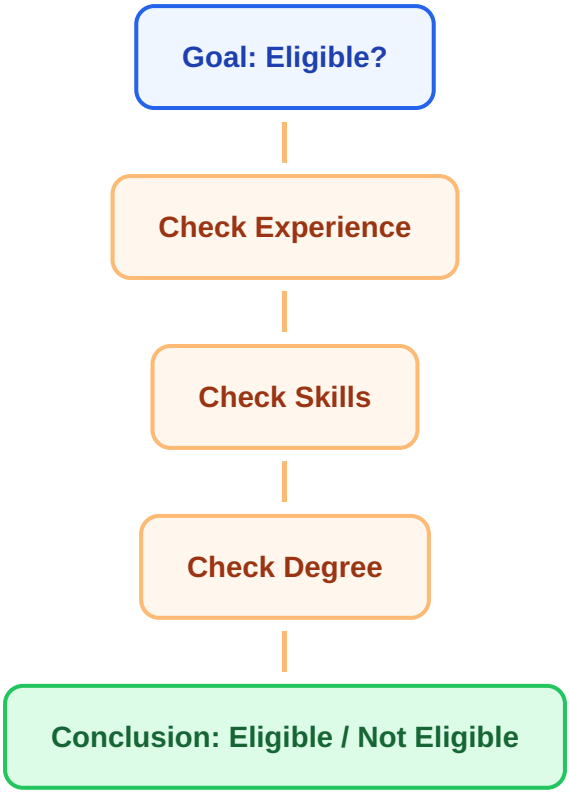
SECTION 10

Backward Chaining

Goal-Driven Inference

Backward chaining starts from a goal and works backward to determine whether the goal can be achieved by verifying required conditions. It asks "Is this candidate eligible?" and then checks each prerequisite.

Inference Flow



SECTION 11

Alpha-Beta Pruning

Purpose: Removing Weak Candidates Early

Alpha-Beta pruning eliminates candidates whose scores fall below a threshold, reducing the search space and computational effort. Weak candidates are pruned early so the system does not waste resources evaluating them further.

Scores & Threshold

CANDIDATE SCORES

92, 89, 55, 43, 90, 96

Pruning Result

Pruned Candidates (Below Threshold)

55 and **43** are pruned because they fall below the threshold of 70.

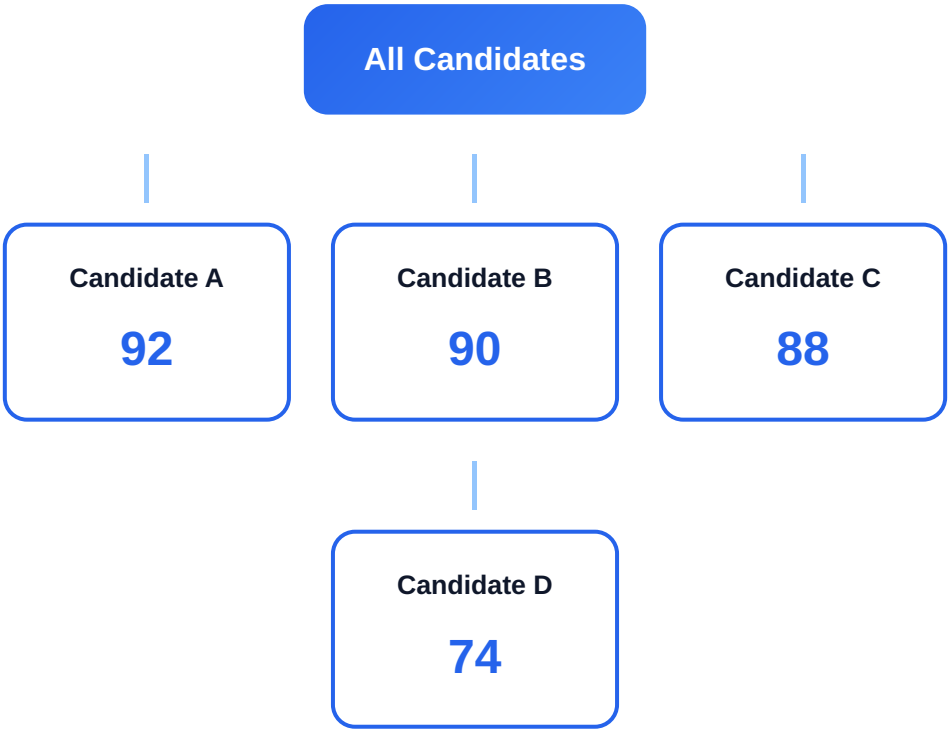
Alpha (α) and Beta (β)

Alpha (α): The minimum acceptable score. Any candidate scoring below alpha is pruned.
Beta (β): The upper bound. If alpha becomes greater than or equal to beta (α ≥ β), pruning occurs because the current branch cannot produce a better result than what has already been found.

SECTION 12

Game Search Tree

The game search tree represents candidate evaluation as a decision tree where the system uses Alpha-Beta pruning to select the best branch (candidate).



Using Alpha-Beta pruning, the system evaluates each candidate branch and selects the best one. Candidate A with a score of 92 is selected as the optimal choice. Candidates below the pruning threshold are eliminated, reducing the search space efficiently.

SECTION 13

Final Ranking

The following table shows the final ranked candidate list produced by the system after applying all classical AI techniques.

Rank	Candidate	Final Score	Status
1	Rahul	92	Selected
2	Priya	90	Selected
3	Arjun	88	Wait List
4	Neha	74	Wait List
5	Karan	55	Rejected

SECTION 14

Conclusion

This project successfully demonstrates that classical AI techniques can be combined into a cohesive pipeline for automated resume shortlisting. Each algorithm contributes a distinct capability to the overall system.

Uninformed Search (BFS/DFS) performs initial eligibility screening by systematically checking all candidate attributes. **Heuristic Search** ranks candidates using a weighted scoring function that evaluates skills, experience, projects, and certifications. **Local Search** iteratively refines the ranking by exploring neighboring solutions to find the optimal candidate order. **Constraint Satisfaction Problems (CSP)** ensure that only candidates meeting all hiring constraints proceed. **Forward Chaining** derives conclusions from known facts in a data-driven manner, while **Backward Chaining** verifies eligibility by working from the goal backward. **Alpha-Beta Pruning** eliminates weak candidates early, and the **Game Search Tree** selects the best candidate.

The resulting system is **rule-based**, **explainable**, **transparent**, and does **not require machine learning**. Every decision can be traced back to specific rules and scores, making it suitable for environments where accountability and interpretability are essential.